MOVIE RECOMMENDATION SYSTEM
End-to-End Data Management Pipeline

Name: Nitu Singh

Roll no. 2025AG05507

Assignment 2 — Execution Evidence Report

GitHub repository: https://github.com/nitusingh27/DM4ML---Assignment-2

Project summary

This individual assignment implements an end-to-end movie recommendation data-management pipeline covering ingestion, validation, preparation, feature management, model training, evaluation and orchestration. The project uses synthetic data consisting of 250 users, 120 movies and 15,000 ratings; it is not the MovieLens dataset.

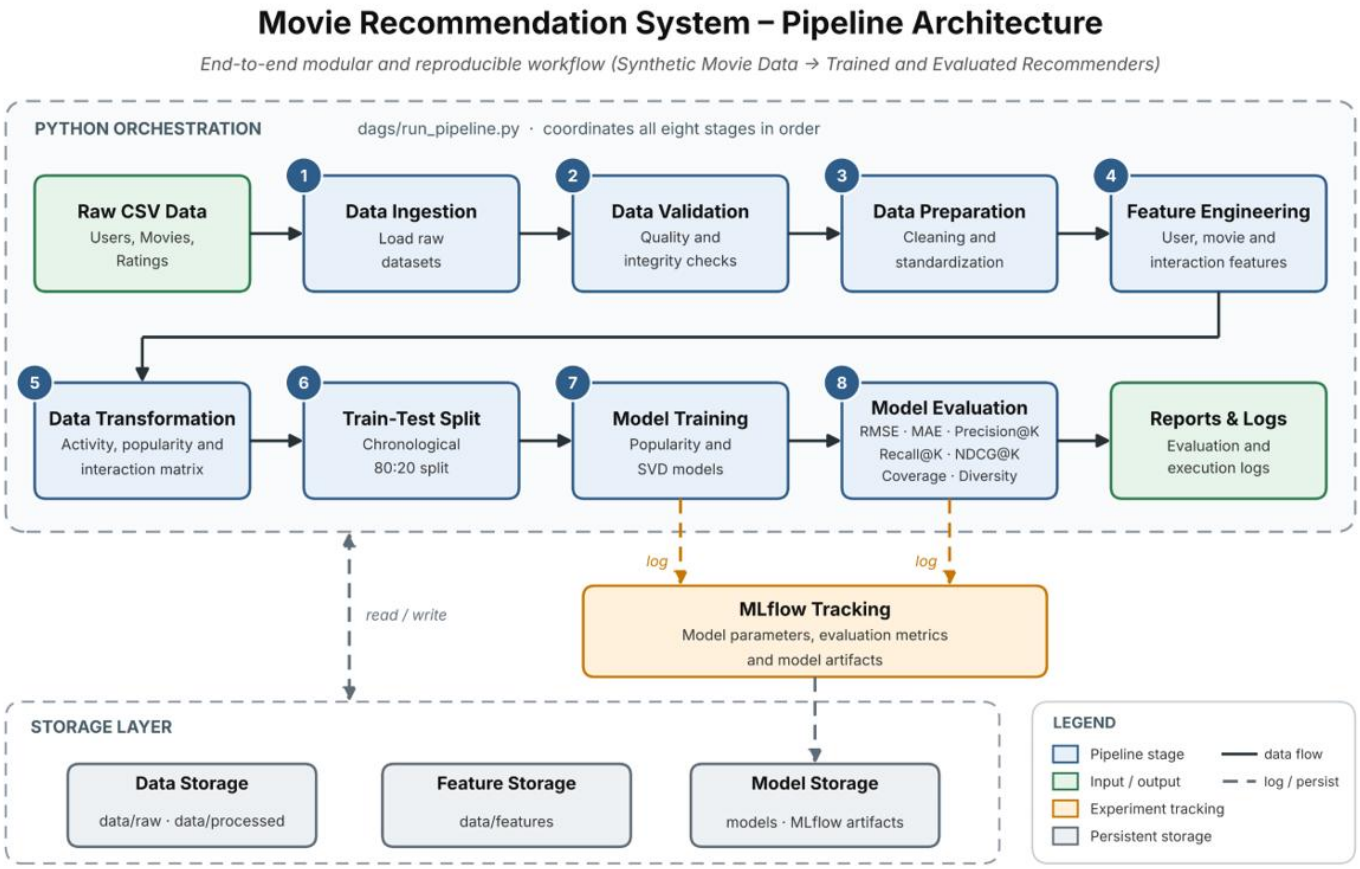
Models: popularity-based recommendation baseline and personalized collaborative filtering using TruncatedSVD. MLflow is used for experiment tracking.

Evidence: Screenshots in this report were captured from the existing Google Colab session. No pipeline rerun was required to prepare this report.

1. Pipeline architecture

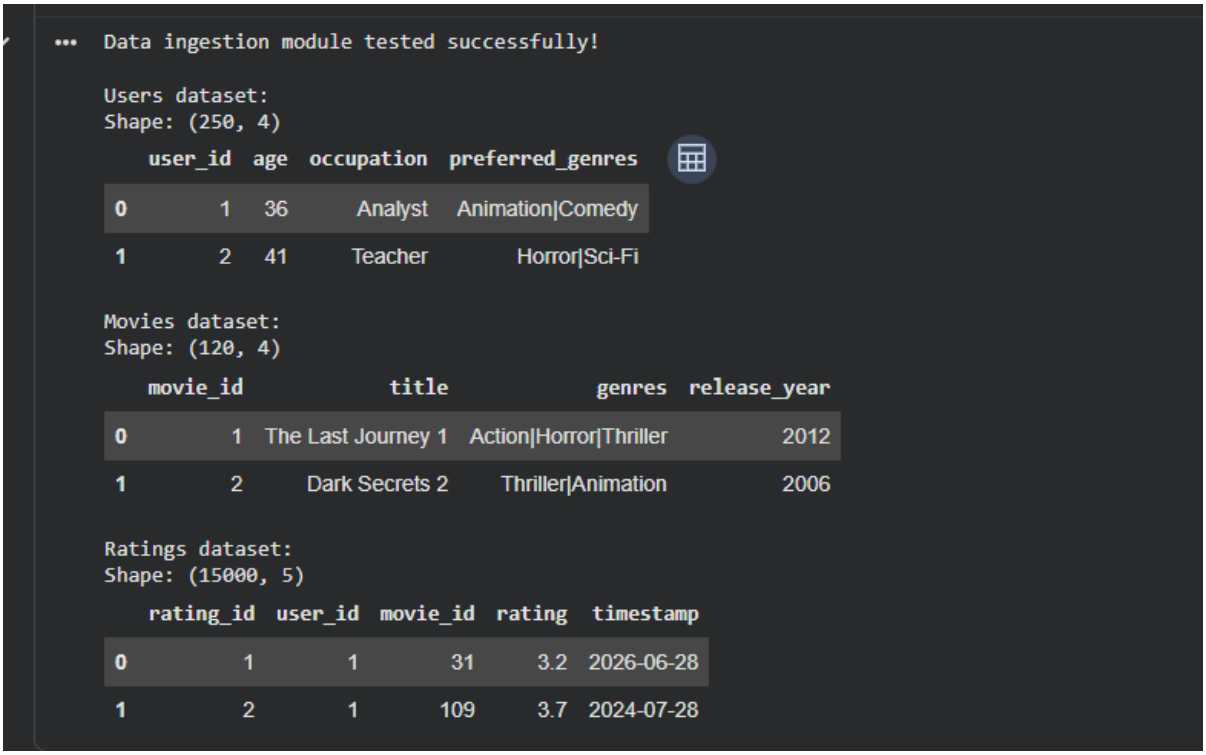
The architecture separates the pipeline into modular stages. Raw datasets are ingested, validated and prepared; feature engineering and transformation produce downstream datasets; recommendation models are trained and evaluated; outputs and run evidence are saved.

Figure 1. End-to-end pipeline architecture.



2. Data ingestion

The ingestion stage loads users, movies and ratings CSV files from the raw data directory. The project dataset contains 250 users, 120 movies and 15,000 rating interactions.



```
Data ingestion module tested successfully!

Users dataset:
Shape: (250, 4)
```

	user_id	age	occupation	preferred_genres
0	1	36	Analyst	Animation|Comedy
1	2	41	Teacher	Horror|Sci-Fi

```
Movies dataset:
Shape: (120, 4)
```

	movie_id	title	genres	release_year
0	1	The Last Journey 1	Action|Horror|Thriller	2012
1	2	Dark Secrets 2	Thriller|Animation	2006

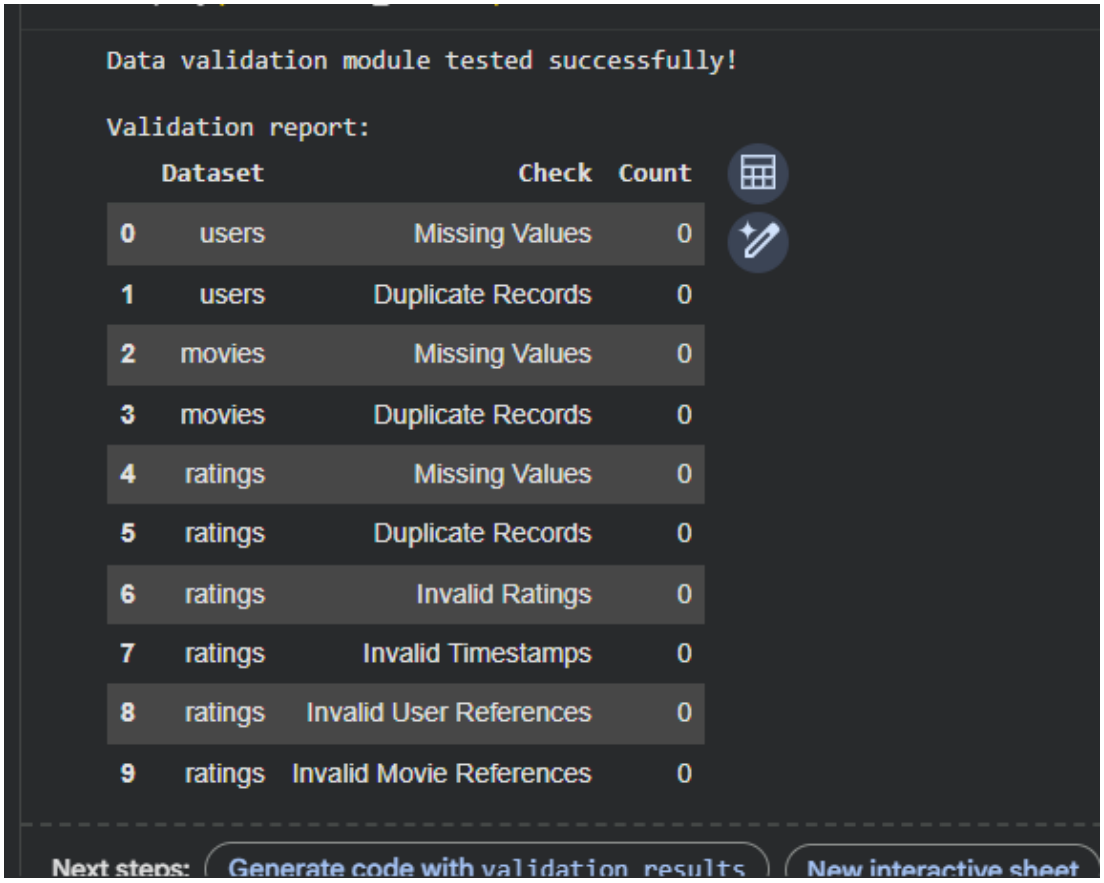
```
Ratings dataset:
Shape: (15000, 5)
```

	rating_id	user_id	movie_id	rating	timestamp
0	1	1	31	3.2	2026-06-28
1	2	1	109	3.7	2024-07-28

Figure 2. Data ingestion output and sample source records.

3. Data validation

The validation stage checks missing values, duplicate records, invalid ratings, timestamps and referential integrity between users, movies and ratings. The captured validation output reports zero issues for the displayed raw dataset.



Data validation module tested successfully!

Validation report:

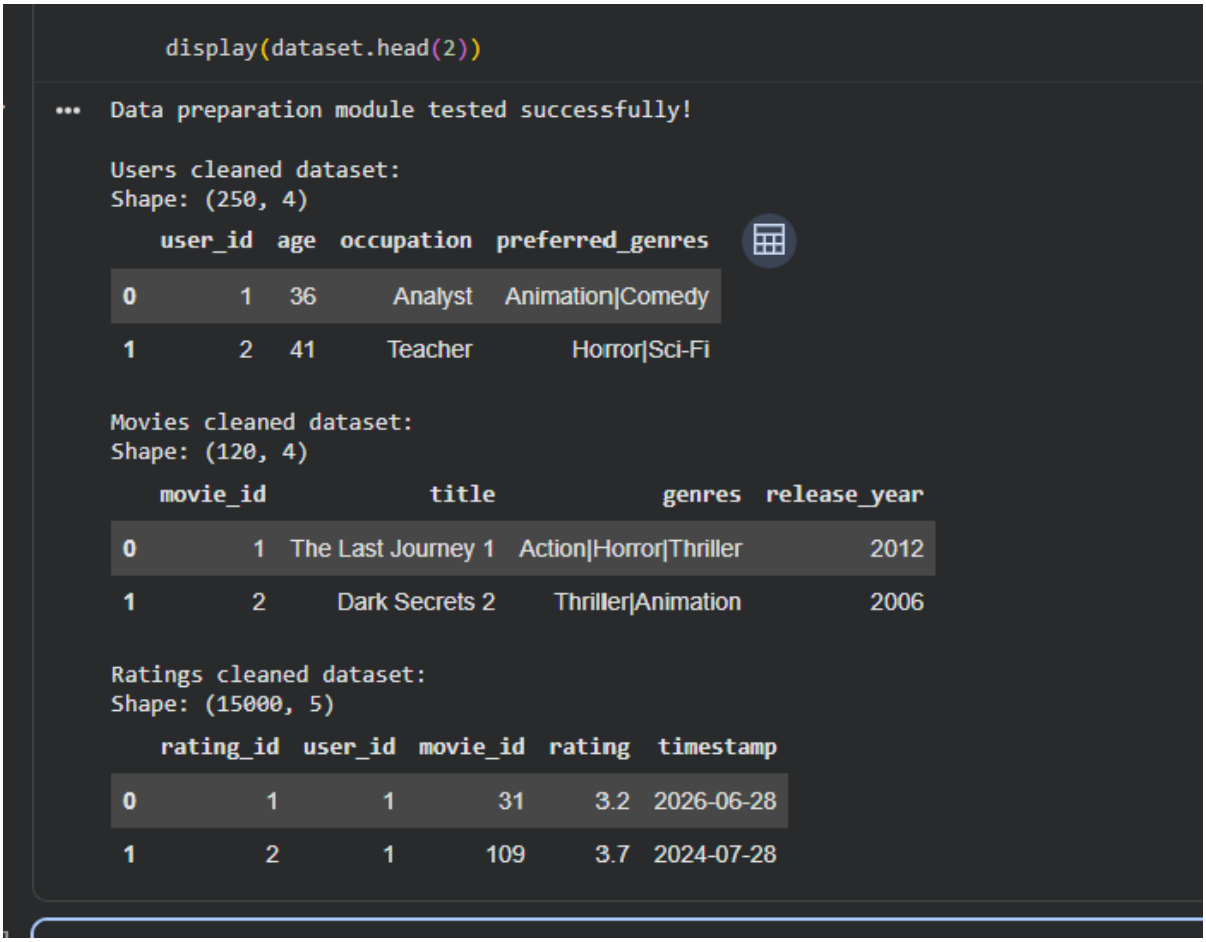
	Dataset	Check	Count
0	users	Missing Values	0
1	users	Duplicate Records	0
2	movies	Missing Values	0
3	movies	Duplicate Records	0
4	ratings	Missing Values	0
5	ratings	Duplicate Records	0
6	ratings	Invalid Ratings	0
7	ratings	Invalid Timestamps	0
8	ratings	Invalid User References	0
9	ratings	Invalid Movie References	0

Next steps: Generate code with validation_results New interactive sheet

Figure 3. Validation report.

4. Data preparation

The preparation stage applies cleaning rules and produces cleaned user, movie and rating datasets for downstream processing.



```
display(dataset.head(2))
```

```
Data preparation module tested successfully!

Users cleaned dataset:
Shape: (250, 4)
```

	user_id	age	occupation	preferred_genres
0	1	36	Analyst	Animation|Comedy
1	2	41	Teacher	Horror|Sci-Fi

```
Movies cleaned dataset:
Shape: (120, 4)
```

	movie_id	title	genres	release_year
0	1	The Last Journey 1	Action|Horror|Thriller	2012
1	2	Dark Secrets 2	Thriller|Animation	2006

```
Ratings cleaned dataset:
Shape: (15000, 5)
```

	rating_id	user_id	movie_id	rating	timestamp
0	1	1	31	3.2	2026-06-28
1	2	1	109	3.7	2024-07-28

Figure 4. Cleaned dataset output.

5. Feature management and transformation

Feature engineering creates separate user, movie and interaction feature datasets. The confirmed feature dataset dimensions are:

Feature dataset	Rows × columns
user_features.csv	250 × 17
movie_features.csv	120 × 12
interaction_features.csv	15,000 × 8

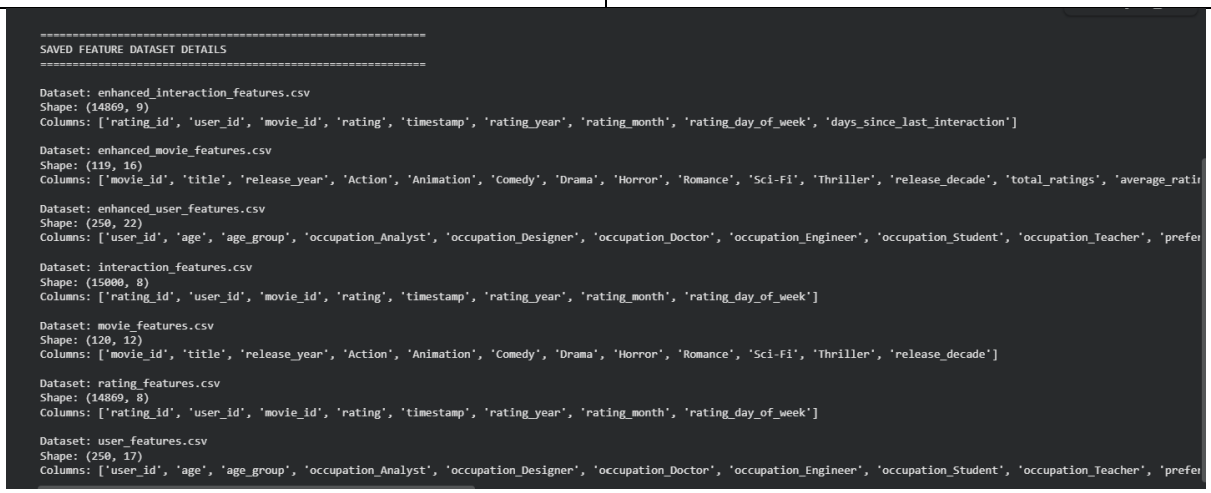


Figure 5. Saved feature dataset details from Colab.

5.1 Derived transformation datasets

The transformation stage also creates user activity summaries, item popularity statistics and a user-item matrix. The model-training output confirms a 250 × 120 user-item matrix.

```
display(user_activity.head())

print("\nTotal users in activity summary:")
print(user_activity["user_id"].nunique())
```

```
User activity summaries created successfully!

User activity dataset shape:
(250, 6)

Sample user activity summaries:
```

	user_id	total_ratings	average_rating	minimum_rating	maximum_rating	last_interaction
0	1	42	3.10	1.4	5.0	2026-03-27
1	2	47	3.24	1.6	5.0	2026-03-03
2	3	48	3.38	1.7	5.0	2026-02-02
3	4	47	3.41	1.6	5.0	2026-04-28
4	5	48	3.30	1.0	5.0	2026-04-09

```
Total users in activity summary:
250
```

Figure 6. User activity summary.

```
Item popularity statistics created successfully!

Item popularity dataset shape:
(119, 5)

Top 10 most rated movies:
```

	movie_id	total_ratings	average_rating	minimum_rating	maximum_rating
1	3	116	3.59	1.4	5.0
16	18	112	3.69	1.0	5.0
88	90	112	3.06	1.1	5.0
48	50	111	3.35	1.0	5.0
33	35	111	3.59	1.8	5.0
67	69	111	3.52	1.0	5.0
7	9	110	3.23	1.3	5.0
57	59	110	3.39	1.3	5.0
72	74	110	3.66	1.7	5.0
103	105	110	3.06	1.1	5.0

```
Total movies with ratings:
119
```

Figure 7. Item popularity statistics.

6. Model training and recommendation outputs

Two recommendation approaches are trained: a popularity-based baseline and personalized collaborative filtering using TruncatedSVD. The Colab training output confirms a popularity model with shape 120 × 3, a user-item matrix with shape 250 × 120, and predicted ratings with shape 250 × 120.

```
96]
0s
from src.training.train import train_recommendation_models

trained_models = train_recommendation_models(train_ratings)

print("Models trained successfully!")

print("\nPopularity model shape:",
      trained_models["popularity_model"].shape)

print("User-item matrix shape:",
      trained_models["user_item_matrix"].shape)

print("Predicted ratings shape:",
      trained_models["predicted_ratings"].shape)

...  Models trained successfully!

Popularity model shape: (120, 3)
User-item matrix shape: (250, 120)
Predicted ratings shape: (250, 120)
```

Figure 8. Model training output and dataset dimensions.

```
display(popular_recommendations)

Popularity-based recommendation model created successfully!

Top 10 recommended movies:
```

	movie_id	title	total_ratings	average_rating
0	3	Love in Paris 3	116	3.59
1	18	City of Dreams 18	112	3.69
2	90	The Last Journey 90	112	3.06
3	50	Dark Secrets 50	111	3.35
4	35	Dark Secrets 35	111	3.59
5	69	Hidden Truth 69	111	3.52
6	9	The Final Mission 9	110	3.23
7	59	The Final Mission 59	110	3.39
8	74	The Lost World 74	110	3.66
9	105	Love in Paris 105	110	3.06

Figure 9. Popularity-based recommendation output.

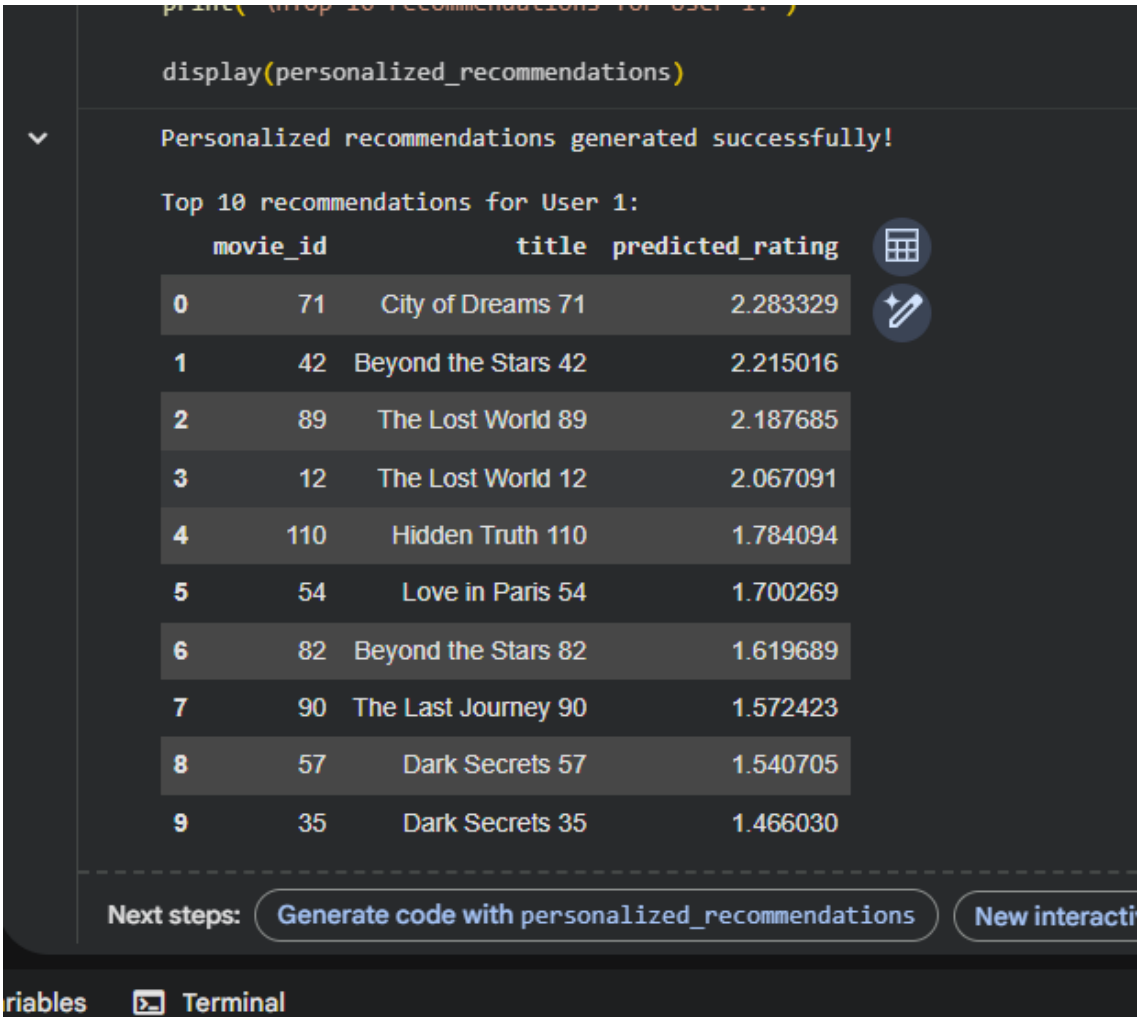
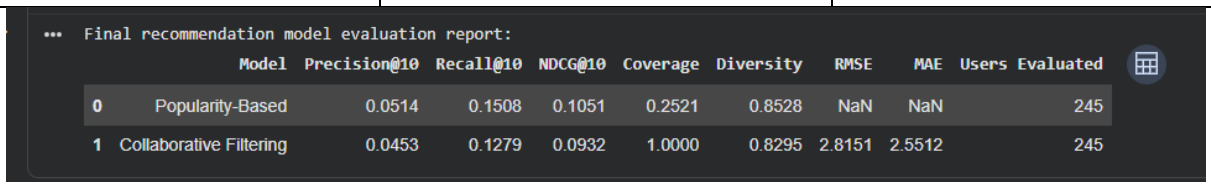


Figure 10. Personalized recommendations for a sample user.

7. Model evaluation

The final/latest evaluation output compares both models at K=10. The project uses rating ≥ 4 as the relevance threshold. RMSE and MAE are reported for collaborative-filtering rating predictions; the popularity baseline is ranking-only, so rating-prediction RMSE and MAE are not applicable.

Metric	Popularity-based	Collaborative filtering
Precision@10	0.0514	0.0453
Recall@10	0.1508	0.1279
NDCG@10	0.1051	0.0932
Coverage	0.2521	1.0000
Diversity	0.8528	0.8295
RMSE	N/A	2.8151
MAE	N/A	2.5512



Final recommendation model evaluation report:

	Model	Precision@10	Recall@10	NDCG@10	Coverage	Diversity	RMSE	MAE	Users Evaluated
0	Popularity-Based	0.0514	0.1508	0.1051	0.2521	0.8528	NaN	NaN	245
1	Collaborative Filtering	0.0453	0.1279	0.0932	1.0000	0.8295	2.8151	2.5512	245

Figure 11. Final model evaluation output captured in Colab.

8. MLflow experiment tracking

MLflow experiment tracking was configured using the project SQLite tracking URI. The experiment Movie_Recommendation_System (ID 1) is listed in the Colab output.

```
MLflow Tracking URI:
sqlite:////content/drive/MyDrive/Movie_Recommendation_Project/mlflow.db

Available MLflow experiments:
Experiment: Movie_Recommendation_System | ID: 1
Experiment: Default | ID: 0
```

Figure 12. MLflow tracking URI and available experiments.

The experiment contains two completed runs: Collaborative_Filtering_Model and Popularity_Based_Model. The output also shows logged parameter and metric fields.

Total MLflow runs: 2

	run_id	status	start_time	end_time	tags.mlflow.runName
0	81549d3c3d904f558dbe2c2b81516026	FINISHED	2026-11-10 13:38:45.798000+00:00	2026-11-10 13:38:46.288000+00:00	Collaborative_Filtering_Model
1	dc2fd67c76f04d2d98a04efc354f4058	FINISHED	2026-11-10 13:38:09.456000+00:00	2026-11-10 13:38:10.941000+00:00	Popularity_Based_Model

```
Logged parameter columns:
['params.n_components', 'params.algorithm', 'params.model_type', 'params.top_n', 'params.number_of_users', 'params.number_of_movies', 'params.ranking_method']

Logged metric columns:
['metrics.explained_variance_ratio', 'metrics.recommendations_generated']
```

Figure 13. Two finished MLflow runs and logged parameter/metric columns.

9. Pipeline orchestration

The project includes a structured Python orchestration script at dags/run_pipeline.py. The saved verification output confirms that orchestration_results.json and orchestration_run.log exist and that the recorded execution status is SUCCESS.

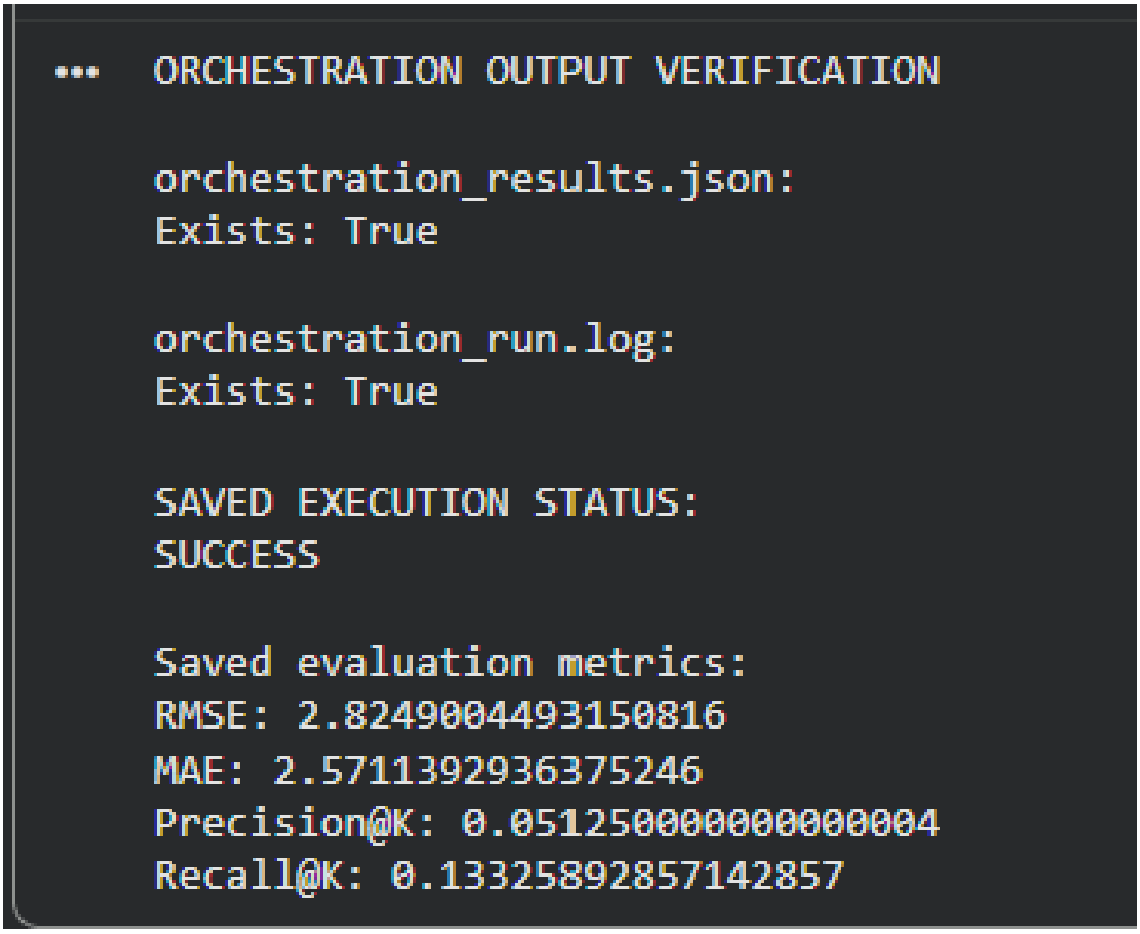


Figure 14. Orchestration output verification from the existing Colab session.

Metric consistency note: The orchestration screenshot records metrics from its saved pipeline execution (RMSE 2.8249 and MAE 2.5711). The final/latest model-comparison screenshot separately reports RMSE 2.8151 and MAE 2.5512. These are retained as distinct captured outputs rather than silently substituted for one another.

The workflow is manually triggered through a structured Python script; no periodic scheduler is configured.

10. Conclusion and limitations

The project demonstrates an end-to-end data-management pipeline for a recommendation system, including ingestion, validation, preparation, feature management, training, evaluation, MLflow tracking and orchestration. The dataset is synthetic. The user-item matrix represents missing interactions as zeros, which is a modelling limitation. The orchestration evidence is from a manually triggered workflow.

GitHub repository: https://github.com/nitusingh27/DM4ML---Assignment-2